

Software Design Document: **Tunki 2D**

Выполнил Сурков Андрей Анатольевич

1. Введение

- **Цель документа:** Предоставить подробное описание проектных решений для 2D игры “Tunki 2D” на Python с использованием Pygame.
- **Область применения:** Документ предназначен для разработчиков, участвующих в создании игры
- **Ссылки:** SRS GDD

2. Общее описание системы

- **Обзор системы:** Игра представляет собой простую танковую аркаду формата PvP Hotseat, позволяющую двум игрокам соревноваться между собой. Задача каждого из игроков - собрать как можно больше монеток за ограниченное время.
- **Архитектурные ограничения:** Игра разрабатывается на Python 3.12 с использованием библиотек pygame 2.6.1 и pillow 11.1.0

3. Архитектурный дизайн

- **Архитектурный подход:** Архитектура системы основана на объектно-ориентированном подходе с использованием основных классов для игровых объектов.
- **Диаграмма архитектуры:**

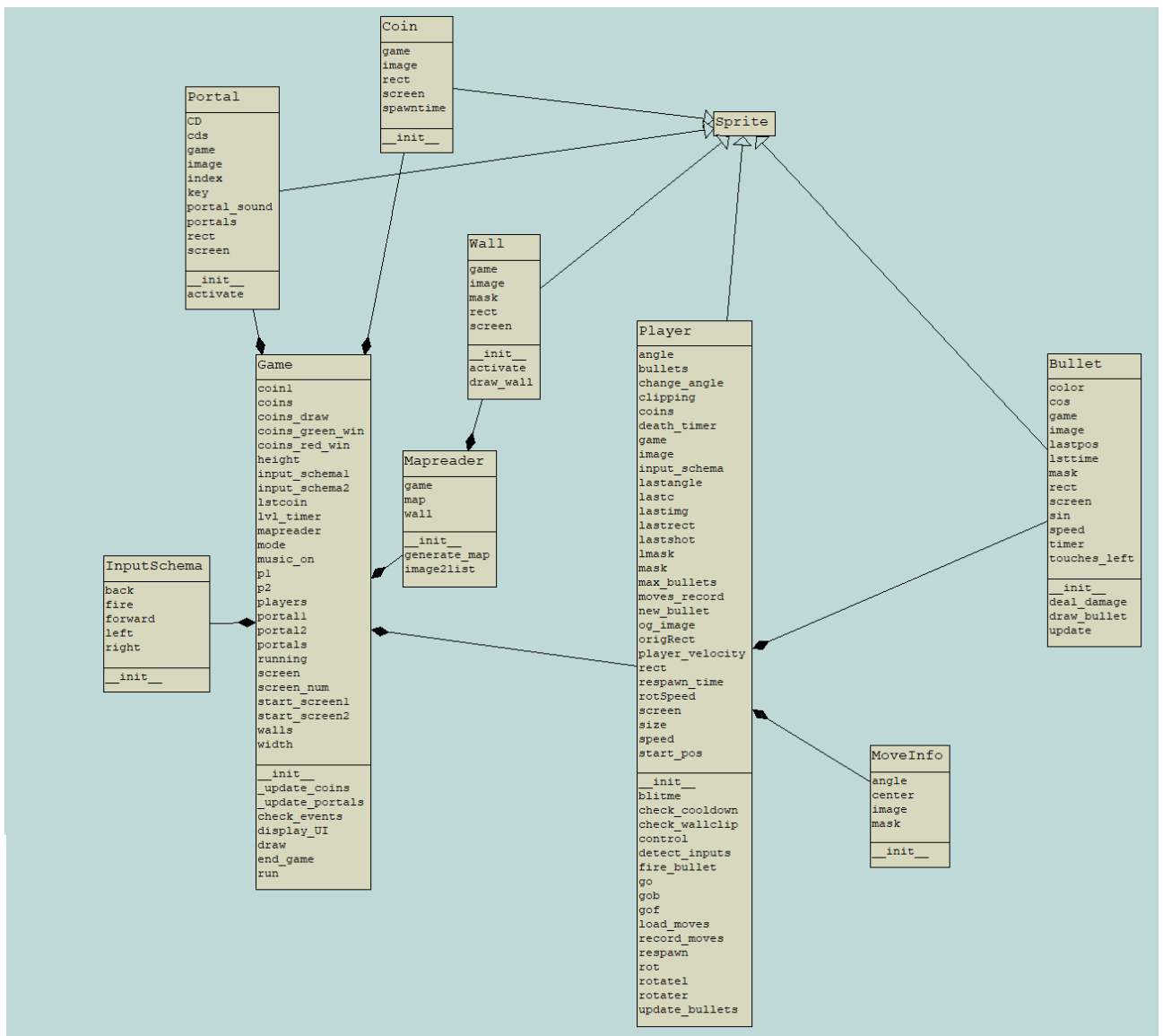


Рис. 1: UML диаграмма

4. Детальное описание компонентов

4.1. Класс Game

Управляет основным игровым циклом и состояниями игры. Методы:

- `_update_coins` - Создает объекты класса `Coin` и управляет ими
- `_update_portals` - Создает объекты класса `Portal` и управляет ими
- `check_events` - Отслеживает игровые события, например, нажатия клавиш пользователем
- `display_UI` - Отрисовка элементов пользовательского интерфейса

- draw - Отрисовка игровых объектов
- end_game - завершение игровой сессии и отображение финального экрана
- run - запуск игрового цикла

4.2. Класс Player

Класс, создающий игрового "персонажа" для обоих игроков. Методы:

- blitme - вывод объекта на экран
- check_cooldown - проверяет боеготовность оружия
- check_wallclip - проверяет, не застрял ли игрок в стене
- control - основной метод, вызывающий остальные
- detect_inputs - регистрирует ввод с клавиатуры и реагирует на него
- fire_bullet - Создание объектов Bullet для реализации выстрелов
- go - движение танка
- rot - поворот танка
- record_moves - сохранение позиции танка в стек и удаление устаревших позиций
- load_moves загрузка позиций из стека в случае, если танк застрял в стене.
- respawn - возрождение танка после смерти
- update_bullets - управление созданными объектами класса Bullet

4.3. Класс Portal

Реализация порталов на карте Методы:

- activate - перенос игрока в следующий портал при касании

4.4. Класс Wall

Реализация стен на карте Методы:

- activate - запрет на движение игрока в случае касания стены
- draw_wall - отображения себя на экране

4.5. Класс Mapreader

Генерация карты на основе файла формата .png Методы:

- `image2list` - превращает карту формата .png в матрицу чисел
- `generate_map` - Создает игровые объекты используя данные из матрицы чисел

4.6. Класс Bullet

Реализация снарядов танков Методы:

- `deal_damage image2list` - определяет эффект, оказываемый на игрока при соприкосновении с пулей
- `draw_bullet` - вывод себя на экран
- `update` - проверка состояния и обновление положения снаряда каждый кадр

5. Дизайн данных

- Все игровые объекты используют класс `Sprite` и хранятся в объектах класса `pygame.sprite.Group`. Работа с этим классом напоминает работу с массивом, однако класс имеет дополнительные функции, связанные с библиотекой `pygame`. Так, например, в переменной `walls` хранятся все созданные объекты класса `Wall`, в `portals` - `Portal`, в `coins` - `Coin`, в `players` - `Player`.
- Игровая карта хранится в файлах игры в виде картинки формата .png, при запуске игры класс `MapReader` обрабатывает этот файл, создавая из него сначала матрицу нулей и единиц, затем создавая игровые объекты и размещая их на карте.
- Класс `MoveInfo` используется в качестве структуры для коротковременного сохранения данных об расположении игрока, которые затем используются классом `Player`
- Классы `InputSchema` используется используется в качестве структуры для создания объекта `Player` с заданной схемой управления. Это необходимо для простого создания нескольких игроков с разными схемами управления.
- Переменные `rect` в разных классах содержат информацию о размерах и положении игровых объектов, Переменные `image` содержат информацию, необходимую для отображения объектов.

- Практически все классы, кроме класса Game и классов-структур, содержат переменную game, которая ссылается на объект класса Game.
- Game является центральным классом, пропускающим через себя или использующим прямо или косвенно данные всех классов.

6. Интерфейсы системы

6.1. Внутренние интерфейсы

Схематическое описание взаимодействия между классами и модулями представлено в Приложении 1 (2)

6.2. Внешние интерфейсы

Описание взаимодействия пользователей с игрой представлено в Приложении 2 (3)

6.3. Макеты экранов

Макеты экранов представлены в Приложении 3 (4)

6.4. UI

Присутствует следующая информация:

- Оставшееся на раунд время.
- Время возрождения каждого из игроков.
- Количество собранных монет для каждого игрока.

7. Требования к производительности

- Время загрузки игры не должно превышать 2 секунд.
- Игра должна потреблять не более 100 МБ оперативной памяти во время выполнения.
- Игра должна поддерживать стабильную частоту кадров не менее 30 FPS.

8. Тестирование и обеспечение качества

8.1. План тестирования

- Тестирование основных механик локально.
- Открытое бета-тестирование с привлечением сторонних пользователей и сбором информации.

8.2. Критерии успешности

- Игра работает без критических ошибок.
- Механики полностью соответствуют описанным в GDD.
- Положительные отзывы группы бета-тестирования.

8.3. Обратная связь

- Учет замечаний для исправлений.
- Сбор идей для потенциального развития.
- Сбор видеоматериала с зафиксированными ошибками работы игры.

9. Требования к среде разработки

9.1. Инструменты разработки:

- Python 3.12+
- pygame 2.6.1+
- pillow 11.1.0+
- PyCharm Community Edition использованы в соответствии с лицензией CC0

10.2. Приложение 2

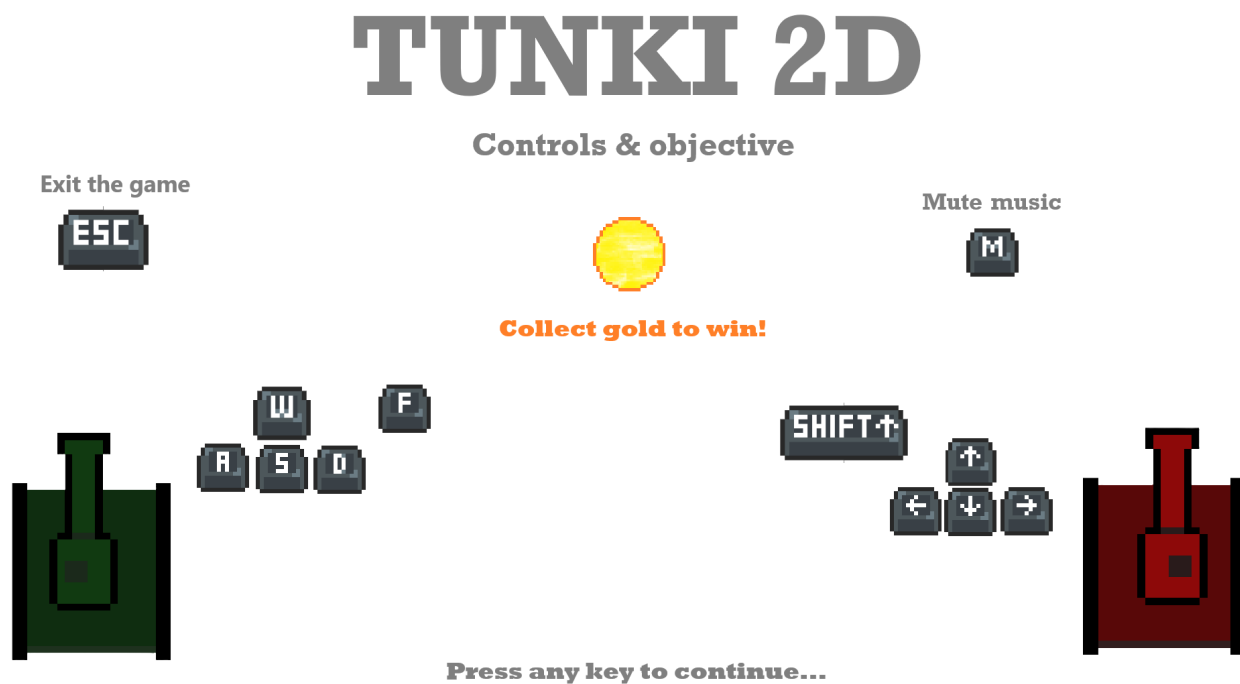


Рис. 3: Управление

10.3. Приложение 3

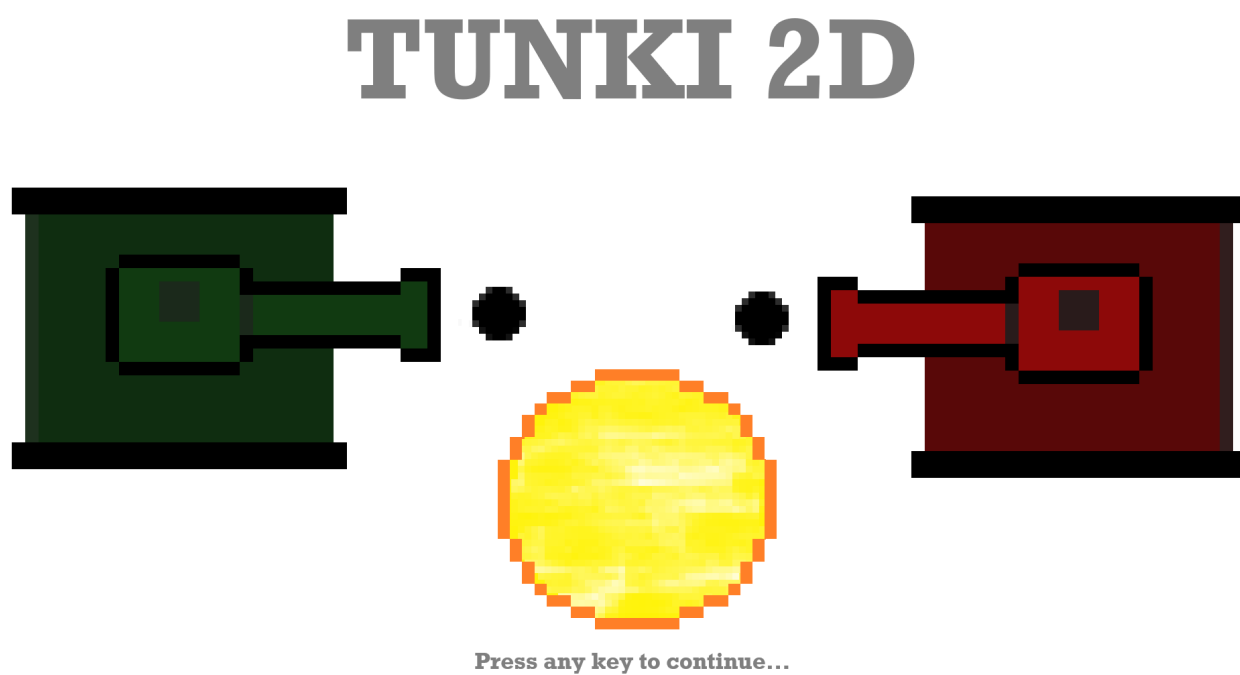


Рис. 4: Стартовый экран

DRAW

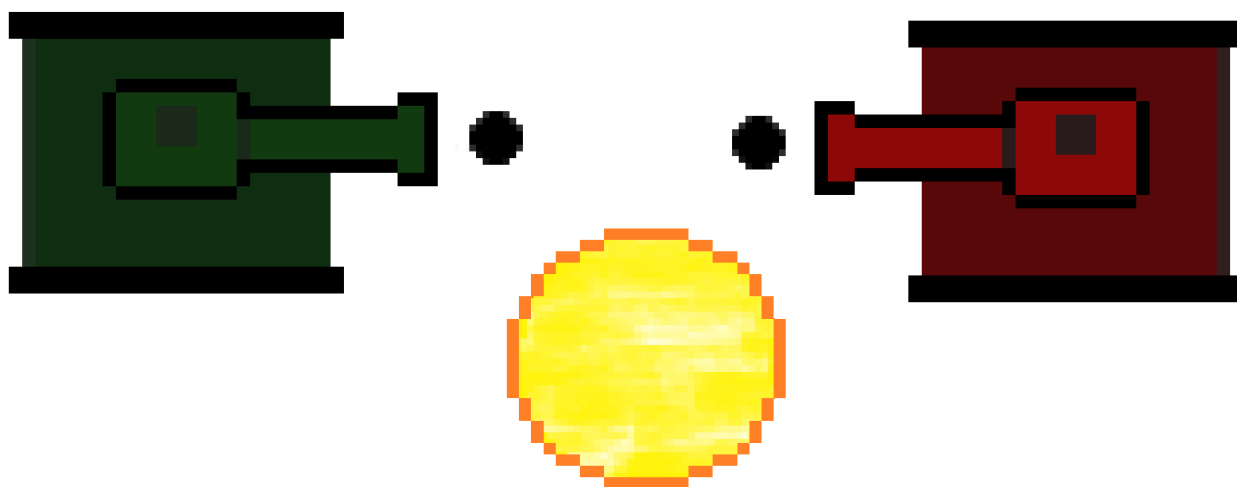


Рис. 5: Экран ничьи

GREEN WINS!

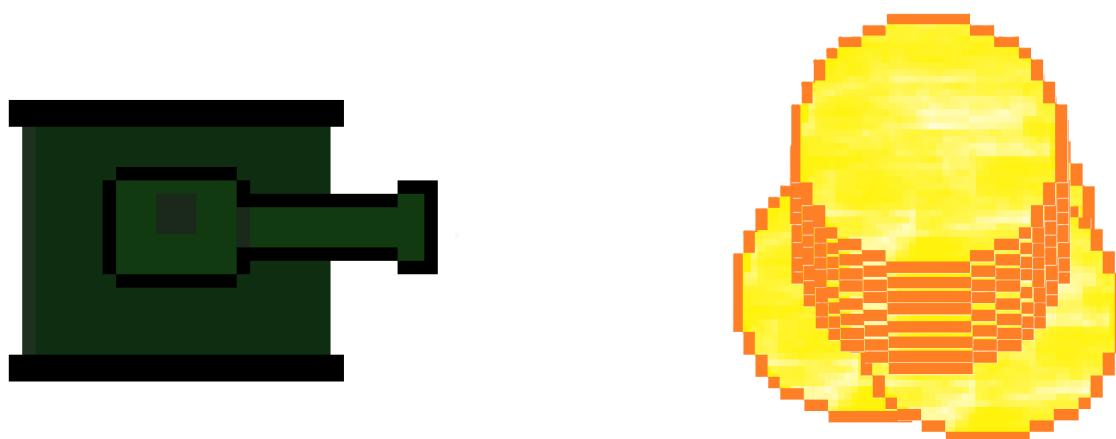


Рис. 6: Экран победы первого игрока

RED WINS!

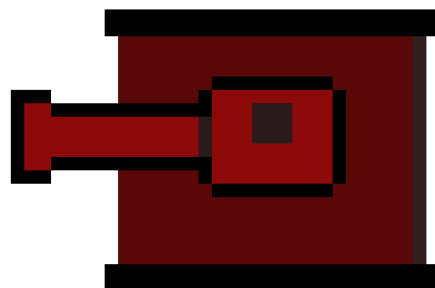
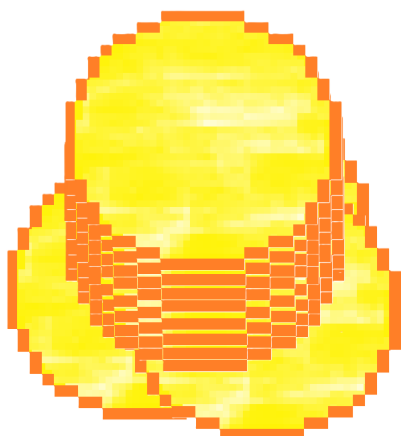


Рис. 7: Экран победы второго игрока

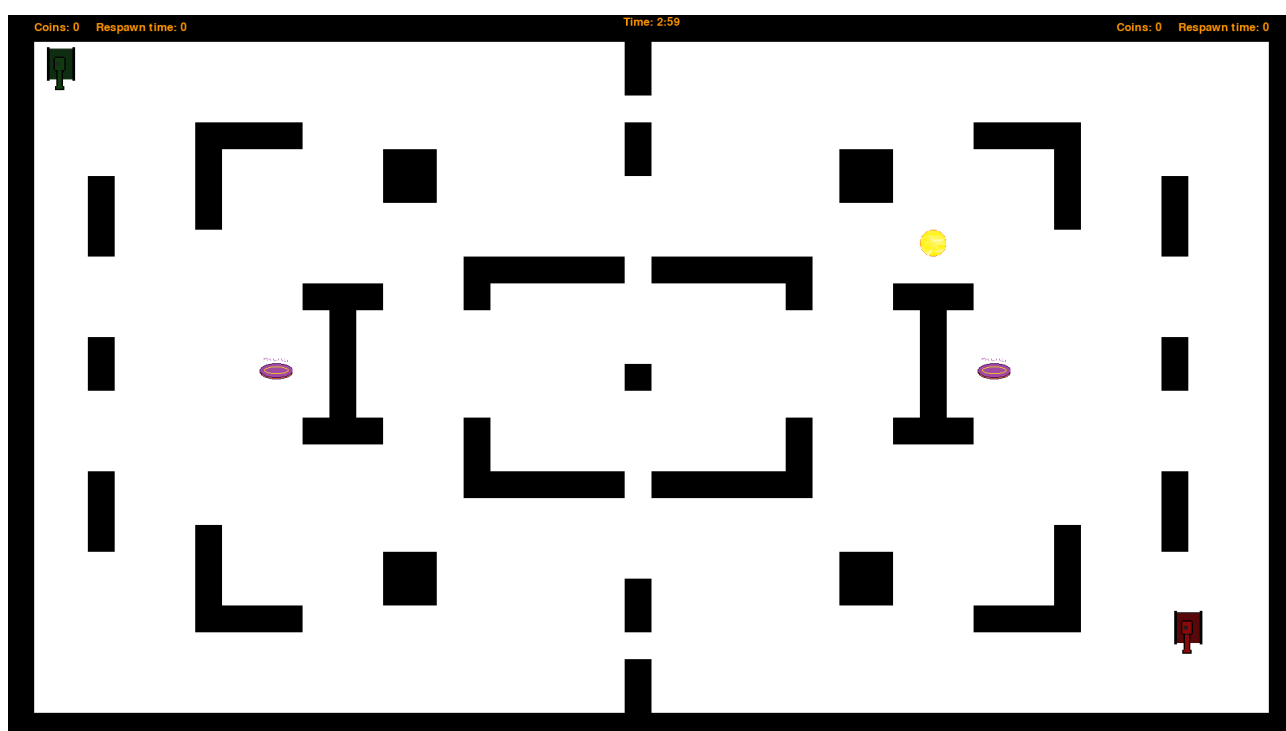


Рис. 8: Отображение уровня в начале игры